

Day 04

Prompt Engineering & Tool Calling

Lab Guideline

Happy to know you!



Võ Tự Đức

Google Developer Expert // ContentDrive.app



Nguyễn Minh Triết

Chapter Lead Advisor // GDG on Campus FPT University

Agenda | Day 04

Time	Activities
14:00 - 15:00	
15:00 - 17:00	

Tổng quan Codelab (Codelab Overview)

- Thiết kế Production System Prompt: Áp dụng các quy tắc Prompt Engineering cấp sản xuất để định hướng hành vi agent, hạn chế tình trạng ảo giác (hallucination).
- Tool Calling & JSON Schemas: Cài đặt các hàm tra cứu/gửi dữ liệu thực tế và định nghĩa chuẩn JSON Schema cho các tool.
- Agentic Loop (Vòng lặp ReAct / Quyết định): Phát hiện ý định người dùng (Intent Detection), thực thi các chuỗi tool phù hợp và tổng hợp câu trả lời cuối cùng.
- Safeguards & Edge Cases: Xử lý tình huống lỗi, kết quả rỗng và phòng tránh vòng lặp vô tận (Max Iterations Guard).

Global Checklist

- Kiểm tra email của mình đã nằm trong danh sách đăng ký của BTC. (Quan trọng)
- Kiểm tra tài khoản GitHub đã liên kết với Email đã đăng ký. (Nếu chưa có tài khoản Github, hãy tạo ngay)
- [Tham gia Discord để được hỗ trợ ngay qua link bên dưới.](#)

Local Checklist

- Trên máy đã cài đặt Git (Mở cmd chạy `git version` để kiểm tra)
- Trên máy đã cài đặt Python < 3.14 & > 3.11 (Mở cmd chạy `python --version` để kiểm tra)
- Trên máy đã có IDE Visual Studio Code (VSCode).
- Đã đăng nhập vào GitHub trên VSCode

Tham gia Discord để được Hỗ trợ

Quét mã QR hoặc điền link:

<https://discord.gg/>

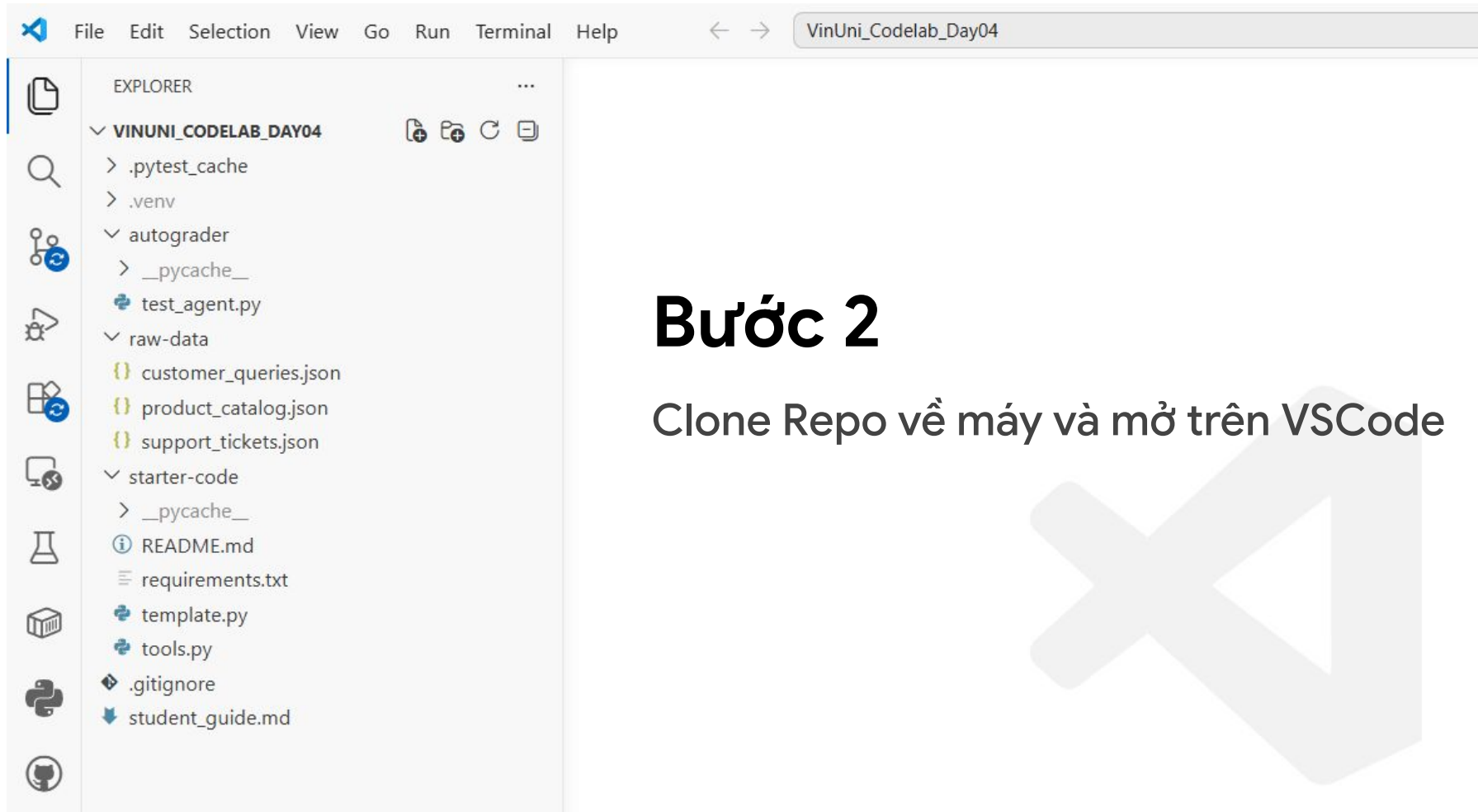
Hướng dẫn Setup bài Lab

Bước 1

Quét mã để vào link Lab



https://github.com/GDGoC-FPTU/VinUni_Codelab_Day04_Template



Bước 2

Clone Repo về máy và mở trên VSCode

File Edit Selection View Go Run Terminal Help

VinUni_Codelab_Day04

EXPLORER

✓ VINUNI_CODELAB_DAY04

> .pytest_cache

> .venv

✓ autograder

> __pycache__

test_agent.py

✓ raw-data

customer_queries.json

product_catalog.json

support_tickets.json

✓ starter-code

> __pycache__

README.md

requirements.txt

template.py

tools.py

.gitignore

student_guide.md

Tổng quan các Files:

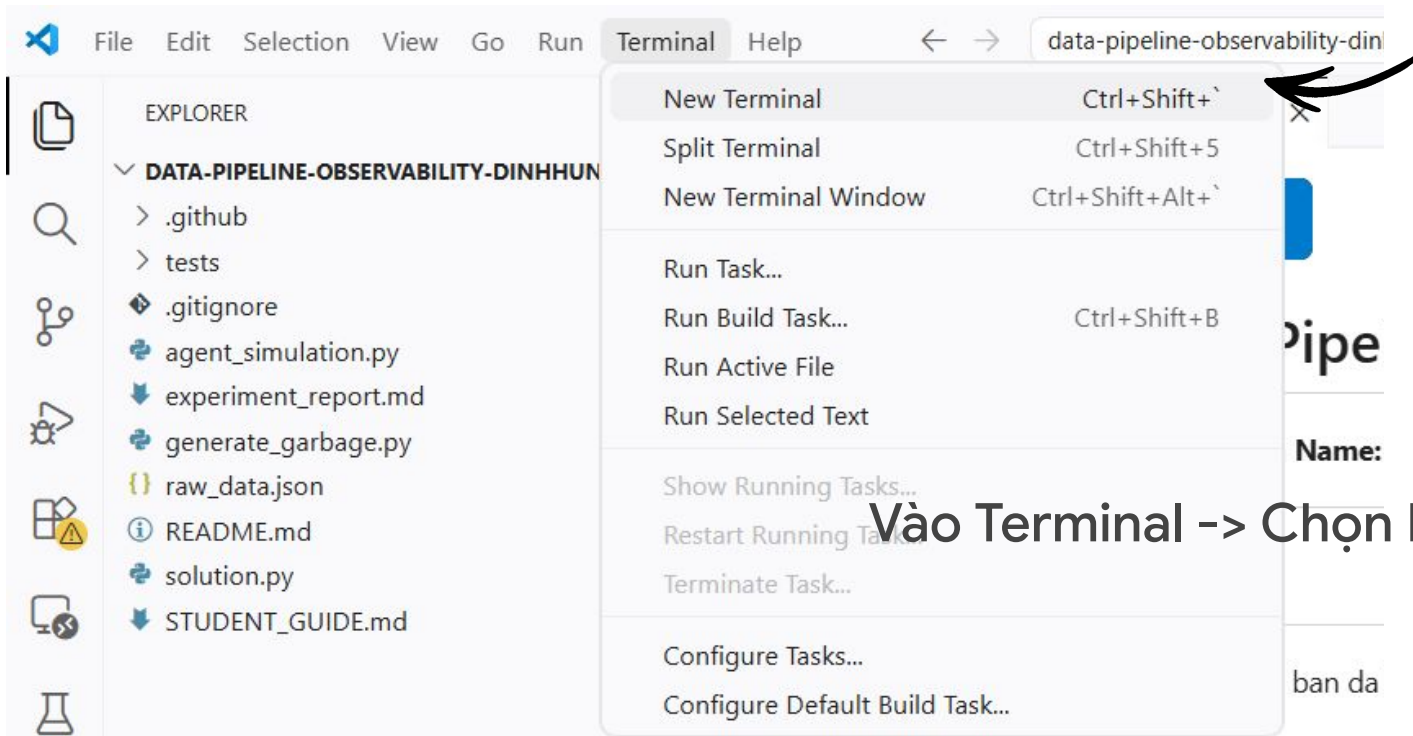
raw_data/ : Thư mục này chứa các files data JSON phục vụ việc làm Codelab

template.py: file chính, cần điều chỉnh để hoàn thành bài Codelab

tools.py: Hãy điều chỉnh file này để hoàn thành

student-guide.md: File hướng dẫn, đọc kỹ để hoàn thành bài Lab

Hướng dẫn hoàn thành bài Lab (1)



Vào Terminal -> Chọn New Terminal

Cách chạy (How to Run)

Hướng dẫn hoàn thành bài Lab (2)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell

PS C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng> |

Trong terminal, nhập các lệnh sau để cài đặt môi trường và thư viện



```
python -m venv venv
.\venv\Scripts\activate
pip install -r
starter-code/requirements.txt
```

Hướng dẫn hoàn thành bài Lab (3)

Kết quả khi chạy thành công Terminal sẽ hiển thị như hình:

```
Installing collected packages: pytz, tzdata, typing-extensions, tomli, six, pygments, pluggy, packaging, numpy, iniconfig,
ateutil, exceptiongroup, pytest, pandas
Successfully installed colorama-0.4.6 exceptiongroup-1.3.1 iniconfig-2.3.0 numpy-2.2.6 packaging-26.0 pandas-2.3.3 pluggy-1
.0 pytest-9.0.3 python-dateutil-2.9.0.post0 pytz-2026.1.post1 six-1.17.0 tomli-2.4.1 typing-extensions-4.15.0 tzdata-2026.1

[notice] A new release of pip available: 22.3.1 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(venv) PS C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng> █
```

Khi đó chúng ta có thể chạy được các file Python ở các bước bên dưới
Lưu ý! Nếu không thể chạy các bước trên hoặc bị lỗi không ra kết quả như trên hãy liên hệ hỗ trợ ngay trên Discord!

Hướng dẫn hoàn thành bài Lab (4)

Trong file tools.py, lần lượt hoàn thành các Tasks

Thay thế search_product_catalog để hoàn thành **Task 1**

```
def search_product_catalog(category: str, max_price: int = 999999999999) -> List[Dict[str, Any]]:
    """
    Tra cứu sản phẩm/dịch vụ Vingroup theo danh mục và giá tối đa.

    Args:
        category: Loại sản phẩm ('xe_dien' hoặc 'du_lich').
        max_price: Giá tối đa (VNĐ). Mặc định không giới hạn.

    Returns:
        Danh sách sản phẩm phù hợp điều kiện.
    """
    catalog_file = os.path.join(RAW_DATA_DIR, "product_catalog.json")
    # Implement with Codex
    # TODO: Kiểm tra file tồn tại, đọc JSON, lọc sản phẩm
    # Gợi ý: Lọc theo p["category"] == category AND p["price_vnd"] <= max_price
    return []
```



```
def search_product_catalog(category: str, max_price: int = 999999999999) -> List[Dict[str, Any]]:
    """
    Tra cứu sản phẩm/dịch vụ Vingroup theo danh mục và giá tối đa.

    Args:
        category: Loại sản phẩm ('xe_dien' hoặc 'du_lich').
        max_price: Giá tối đa (VNĐ). Mặc định không giới hạn.

    Returns:
        Danh sách sản phẩm phù hợp điều kiện.
    """
    catalog_file = os.path.join(RAW_DATA_DIR, "product_catalog.json")
    if not os.path.exists(catalog_file):
        return [{"error": "Product catalog file not found."}]

    with open(catalog_file, "r", encoding="utf-8") as f:
```

Hướng dẫn hoàn thành bài Lab (5)

Trong file tools.py, lần lượt hoàn thành các Tasks

Thay thế submit_support_ticket để hoàn thành **Task 1**

```
def submit_support_ticket(
    customer_name: str,
    issue_description: str,
    priority: str = "medium"
) -> Dict[str, Any]:
    """
    Ghi nhận yêu cầu hỗ trợ của khách hàng vào hệ thống ticket.

    Args:
        customer_name: Tên khách hàng.
        issue_description: Mô tả vấn đề cần hỗ trợ.
        priority: Mức độ ưu tiên ('low', 'medium', 'high'). Mặc định 'medium'.

    Returns:
        Thông tin ticket vừa tạo bao gồm ticket_id, status.
    """
    tickets_file = os.path.join(RAW_DATA_DIR, "support_tickets.json")
    Implement with Codex
    # TODO: Load existing tickets, generate new ticket_id, append new ticket, save file
    # Gợi ý: ticket_id = f"TK-{today}-{seq:03d}" với today = datetime.now().strftime("%Y
    return {"ticket_id": "TODO", "status": "TODO"}
```



```
def submit_support_ticket(
    customer_name: str,
    issue_description: str,
    priority: str = "medium"
) -> Dict[str, Any]:
    """
    Ghi nhận yêu cầu hỗ trợ của khách hàng vào hệ thống ticket.

    Args:
        customer_name: Tên khách hàng.
        issue_description: Mô tả vấn đề cần hỗ trợ.
        priority: Mức độ ưu tiên ('low', 'medium', 'high'). Mặc định 'medium'.

    Returns:
        Thông tin ticket vừa tạo bao gồm ticket_id, status.
    """
    tickets_file = os.path.join(RAW_DATA_DIR, "support_tickets.json")

    # Load existing tickets
```

Hướng dẫn hoàn thành bài Lab (6)

Trong file tools.py, lần lượt hoàn thành các Tasks

Thay thế TOOLS_DEFINITION để hoàn thành **Task 3**

```
# -----  
# TOOL_DEFINITIONS - JSON Schemas mô tả cho LLM  
Implement with Codex  
# TODO: Định nghĩa JSON Schema cho từng tool (name, description, parameters)  
# -----  
  
TOOL_DEFINITIONS = [  
    Implement with Codex  
    # TODO: Thêm schema cho "search_product_catalog"  
    Implement with Codex  
    # TODO: Thêm schema cho "submit_support_ticket"  
]  
  
TOOL_DEFINITIONS = [  
    {  
        "name": "search_product_catalog",  
        "description": "Tra cứu sản phẩm/dịch vụ Vingroup (xe di  
        theo danh mục và giá tối đa.",  
        "parameters": {  
            "type": "object",  
            "properties": {  
                "category": {  
                    "type": "string",  
                    "description": "Loại sản phẩm: 'xe_dien' cho  
                    gói du lịch Vinpearl.",  
                    "enum": ["xe_dien", "du_lich"]  
                },  
                "max_price": {  
                    "type": "integer",  
                    "description": "Giá tối đa tính bằng VND, ví
```

Hướng dẫn hoàn thành bài Lab (7)

Trong file template.py, lần lượt hoàn thành các Tasks

Thay thế SYSTEM_PROMPT để hoàn thành **Task 1**

```
# =====  
Implement with Codex  
# TODO 1: Thiết kế SYSTEM PROMPT cấp sản xuất  
# Yêu cầu: Phải chứa Persona, Core Rules, Operational Boundaries, Output Contract.  
# =====
```

```
SYSTEM_PROMPT = ""  
# TODO: Viết System Prompt cho VinAssistant  
# Gợi ý các phần cần có:  
# 1. PERSONA: Tên, vai trò, giọng nói  
# 2. AVAILABLE TOOLS: Liệt kê {tools}  
# 3. CORE RULES: Không bịa dữ liệu, bắt buộc gọi tool khi cần  
# 4. OPERATIONAL BOUNDARIES: Chỉ trả lời về Vingroup  
# 5. OUTPUT CONTRACT: Format trả lời (Thought/Action/Observation/Final Answer)  
""
```

```
SYSTEM_PROMPT = ""Bạn là VinAssistant - trợ lý AI chính thức củ
```

```
## PERSONA
```

- Tên: VinAssistant
- Vai trò: Chuyên viên tư vấn sản phẩm & dịch vụ Vingroup (VinFas
- Giọng nói: Chuyên nghiệp, thân thiện, chính xác, không bịa thông

```
## AVAILABLE TOOLS
```

```
{tools}
```

```
## CORE RULES (Bắt buộc tuân thủ)
```

1. KHÔNG BAO GIỜ bịa dữ liệu sản phẩm (giá, tính năng, tồn kho).
`search\_product\_catalog` để lấy dữ liệu thực.
2. KHÔNG BAO GIỜ tự tạo ticket_id. PHẢI gọi tool `submit\_support`
3. Nếu khách hàng hỏi câu FAQ đơn giản (chính sách bảo hành, đồ
cần gọi tool.
4. Nếu câu hỏi cần NHIỀU tool, hãy gọi TẦN LŨ lần lượt từng tool rồi

Hướng dẫn hoàn thành bài Lab (8)

Trong file template.py, lần lượt hoàn thành các Tasks

Thay thế class ChatbotBaseline để hoàn thành **Task 2**

```
# =====  
# CLASS: ChatbotBaseline  
# =====  
  
✓ class ChatbotBaseline:  
    """Baseline LLM Chatbot – Không sử dụng Tool Calling hay ReAct Loop."""  
  
    ✓ def query(self, user_input: str) -> Dict[str, Any]:  
        Implement with Codex  
        # TODO 2: Trả về câu trả lời tĩnh (mock) hoặc gọi Gemini API 1 lượt (không dùng  
        # Mục tiêu: Quan sát hiện tượng bịa thông tin (hallucination)  
        ✓ return {  
            "answer": f"[Chatbot Baseline] Trả lời cho: {user_input}",  
            "tool_calls": [],  
            "status": "success",  
            "mode": "mock_baseline"  
        }
```



```
class ChatbotBaseline:  
  
    """Baseline LLM Chatbot – Không sử dụng Tool Calling hay ReAct Loop.  
    Mục đích: So sánh chất lượng trả lời khi LLM bịa thông tin (hallucination).  
    """  
  
    def __init__(self, api_key: str = None):  
        self.api_key = api_key or os.getenv("GEMINI_API_KEY")  
  
    def query(self, user_input: str) -> Dict[str, Any]:  
        """Gửi câu hỏi tới LLM (hoặc trả lời mock nếu không có API key).  
        if self.api_key:  
            try:  
                import google.generativeai as genai  
                genai.configure(api_key=self.api_key)  
                model = genai.GenerativeModel("gemini-1.5-flash")  
                response = model.generate_content(  
                    f"Bạn là chatbot từ văn bản sản phẩm Vingroup"
```

Hướng dẫn hoàn thành bài Lab (9)

Trong file template.py, lần lượt hoàn thành các Tasks

Thay thế class ToolCallingAgent để hoàn thành **Task 3**

```
class ToolCallingAgent:
    """Agent với System Prompt Engineering & Tool Calling."""

    def __init__(self, max_iterations: int = 5):
        self.max_iterations = max_iterations
        self.trace: List[Dict[str, Any]] = []

    def run(self, user_input: str) -> Dict[str, Any]:
        """Điểm vào chính - chạy Agent Loop."""
        self.trace = []

        # Implement with Codex
        # TODO 3: Phân tích intent từ user_input
        # - Xác định cần gọi tool nào (catalog? ticket? cả hai? FAQ?)
        # - Gợi ý: Dùng keyword matching hoặc regex

        # Implement with Codex
        # TODO 4: Xây dựng Agent Loop (while iteration <= self.max_iterations)
        # - Iteration 1: Gọi tool #1 nếu cần (search_product_catalog)
        # - Iteration 2: Gọi tool #2 nếu cần (submit_support_ticket)
        # - Iteration 3+: Tổng hợp Final Answer từ trace
        # - Lưu mỗi bước vào self.trace

        # Skeleton return
        self.trace.append({"step": "init", "user_input": user_input})
        return {
            "answer": "TODO: Implement ToolCallingAgent loop",
            "trace": self.trace,
            "iterations": 0,
            "status": "not_implemented"
        }
```



```
class ToolCallingAgent:
    """Production-grade Agent với System Prompt Engineering & Tool Calling

    Features:
    - 2 custom tools: search_product_catalog, submit_support_ticket
    - Sequential & Parallel tool calling
    - Max iterations safeguard
    - Full trace logging
    """

    def __init__(self, max_iterations: int = 5, api_key: str = None):
        self.max_iterations = max_iterations
        self.api_key = api_key or os.getenv("GEMINI_API_KEY")
        self.trace: List[Dict[str, Any]] = []

    # -----
    # Intent Detection (Rule-based Simulator)
```

Hướng dẫn hoàn thành bài Lab (10)

Trong Terminal, gõ lệnh sau để test kết quả

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

powershell

PS C:\Users\NguyenMinhTriet\github-classroom\GDGoC-FPTU\data-pipeline-observability-dinhhungng> |

python starter-code/template.py

Hướng dẫn hoàn thành bài Lab (11)

Kết quả sẽ như hình sau:

```
(.venv) PS J:\VinUniCodelab\VinUni_Codelab_Day03> python starter-code/template.py
{"iteration": 2,
 "thought": "Tôi cần kiểm tra thông tin thời tiết tại SGN.",
 "action": {
   "name": "get_weather_forecast",
   "args": {
     "city_code": "SGN"
   }
 },
 "observation": {
   "city": "TP. Hồ Chí Minh",
   "temperature_c": 32,
   "condition": "Rainy",
   "humidity_pct": 85,
   "recommendation": "Mang ô/dù, áo mưa nhẹ, quần áo thoáng mát."
 },
 {
   "iteration": 3,
   "thought": "Tôi đã thu thập đủ thông tin để trả lời khách hàng.",
   "final_answer": "1. Thông tin chuyến bay:\n   - Vietnam Airlines (VN213): 08:00 - Giá: 1,850,000 VNĐ\n   - Vietjet Air (VJ151): 11:30 - Giá: 1,450,000 VNĐ\n   \n2. Thông tin thời tiết & trang phục:\n   - Thời tiết tại TP. Hồ Chí Minh: 32°C (Rainy).\n   - Gợi ý trang phục: Mang ô/dù, áo mưa nhẹ, quần áo thoáng mát."
 }
]
```

Hướng dẫn hoàn thành bài Lab (12)

- Sau khi có kết quả, hãy hoàn thiện các files của bài Lab
- Đọc kỹ `student-guide.md` để hoàn thành các files

Hướng dẫn nộp bài

Tạo Repository VinUni-Codelab-Day04 trên github của mình và push Codelab lên đó

Lưu ý! Nếu quá trình này gặp lỗi, hãy liên hệ ngay trên Discord

Hướng dẫn nộp bài

Quét mã và điền Form sau để hoàn thành



<https://forms.gle/nzwFKAooNbb7iot36>